

A FRIENDLY FIELD GUIDE

How Max Agency Works

A robot software company that builds projects on its own — explained from scratch, with pictures.



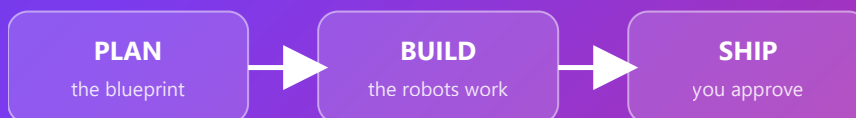
Built for total beginners



Lots of diagrams



Real examples



What's inside

Read it front to back, or jump to whatever you're curious about.

1	The 30-second version	the whole thing in one breath
2	The big idea	a company run by robots
3	Meet the team	five roles, five jobs
4	The three places things live	engine, copy, workshop
5	The sticky-note board	how robots talk: GitHub + labels
6	The life of a task	from idea to finished, step by step
7	A real example, start to finish	one job, fully traced
8	The two engines	Hermes & Claude Code
9	When things go wrong	the safety nets
10	Glossary	every weird word, explained
11	One-page cheat sheet	stick it on your wall

1 · The 30-second version

THE WHOLE IDEA IN ONE BREATH

Max Agency is a **team of AI robots that act like a small software company**. You hand them a goal ("write me a book", "build me an app"). One robot writes a **plan**, another **checks the plan**, a "manager" robot **chops the plan into tasks** and pins them to a shared **to-do board**, and **worker robots** pick up tasks and do them. Every finished piece of work is **checked by a different robot** before **you** give the final yes. The robots never sleep — they wake up every minute, look at the board, and move things forward.

They plan

An Architect robot turns your goal into a written plan. A CTO robot reviews it like a strict teacher.

They build

A manager robot hands out tasks. Worker robots write the actual files and propose changes.

You approve

Nothing important happens without a human. **Only you** press the final "merge" button.

WHY THIS IS COOL

You describe *what* you want once. The robots handle the *how* — splitting work, doing it, checking each other, and asking you only when they truly need a decision.

2 · The big idea

Imagine a tiny company where every employee is an AI, and instead of meetings they use sticky notes.

ANALOGY: THE KITCHEN BRIGADE

Think of a restaurant kitchen. The **head chef** writes the menu (the plan). A **food critic** tastes a sample and says yes/no (the review). The **kitchen manager** calls out orders and pins tickets to the rail. **Cooks** grab tickets, cook the dish, and put it on the pass. Nothing leaves the kitchen until it's checked. Max Agency is that kitchen — but the "dishes" are pieces of a software or writing project, and the "tickets" are tasks on a website called GitHub.

The two things that make it work

1. A shared board everyone can see

All the robots read and write to the same online to-do board (GitHub Issues). Because the board is the single source of truth, a robot can "forget everything", wake up, read the board, and know exactly what to do next. No robot has to remember anything in its head.

2. Colored labels that act like traffic lights

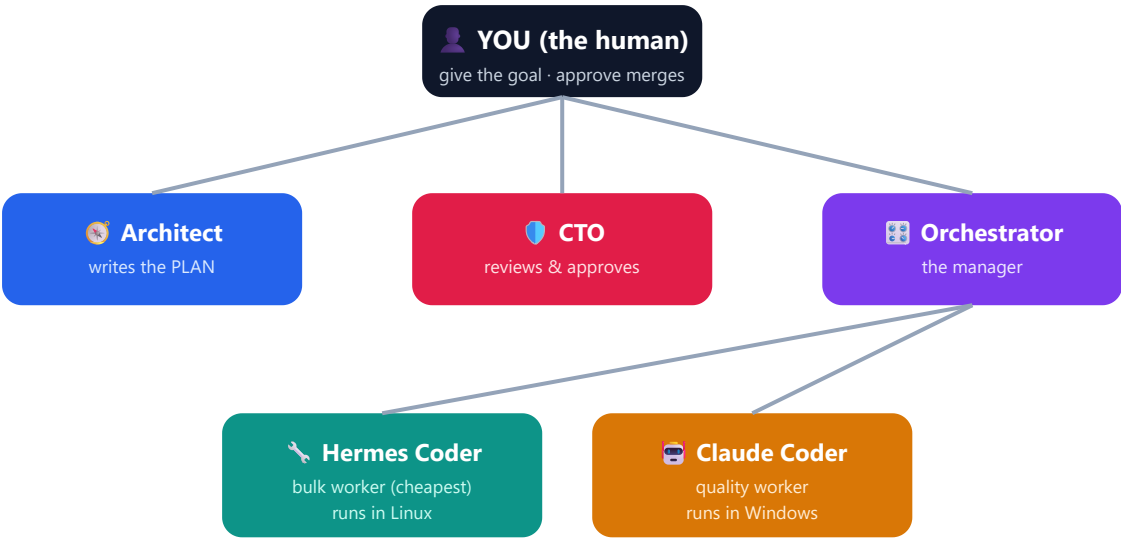
Each task wears small colored **labels**. Labels say *who* should do it, *what kind* of job it is, and *what stage* it's at. Robots only grab tasks whose labels match their job — like a cook only taking tickets for their station.

EXAMPLE

A task labelled `ready` + `assigned:hermes-coder` + `role:coder` means: "This is ready to start, and the Hermes worker-robot should do it." Any other robot ignores it.

3 · Meet the team

Five roles. Each one is an AI given a strict job description and told to stay in its lane.



Who answers to whom. The Orchestrator hands work to the coders; the CTO checks it; you sit on top.

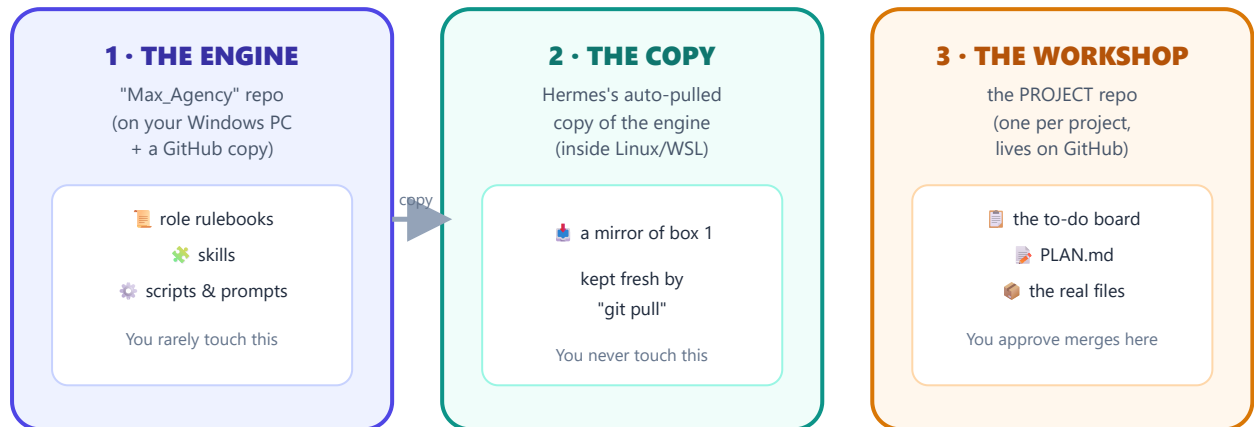
Role	Think of them as...	What they actually do	Never allowed to...
Architect	The blueprint designer	Turns your goal into <code>PLAN.md</code> — a list of phases and tasks with dependencies.	write the actual project files
CTO	The strict examiner	Reviews the plan and every finished piece of work. Says <code>APPROVED</code> or <code>CHANGES REQUIRED</code> .	write code, or approve its own work
Orchestrator	The shift manager	Chops the plan into tasks, hands them out, chases progress, asks you to merge.	write code, or merge things itself
Hermes Coder	The fast, cheap builder	Does the bulk of the building — the many routine tasks.	review its own work
Claude Coder	The careful builder	Handles the trickier, quality-sensitive tasks. Can also <i>act</i> as Architect or CTO when wearing that hat.	review its own work

THE GOLDEN SAFETY RULE

The robot that **builds** a thing is never the same robot that **approves** it, and a robot is **never** allowed to merge — that's your job. This "separation of duties" is what stops the robots from rubber-stamping their own mistakes.

4 · The three places things live

One of the most confusing things at first: there are **three separate copies of things**. Here's the map.



Robots read the rules from box 1/2, but do all the actual work in box 3 (the workshop).

THE #1 BEGINNER MISTAKE

Mixing up the **engine** (the toolkit that powers the robots) with the **workshop** (the actual project being built). They are different GitHub repos. Never copy files between them. The engine is the same for every project; you get a fresh workshop for each new project.

5 · The sticky-note board

Robots can't chat in a hallway. Instead they leave notes on a shared board — **GitHub Issues** — and read the colored **labels** on each note.

Anatomy of one task (a GitHub "issue")

#7 · phase-1/1.6: Write the style guide

"Create bible/STYLE.md following Chicago rules + project tweaks."

in-progress

assigned:hermes-coder

role:coder

phase:1

🔗

 Depends-on: #2 ·

👤

 Assignee: (claimed by a worker) ·

🔗

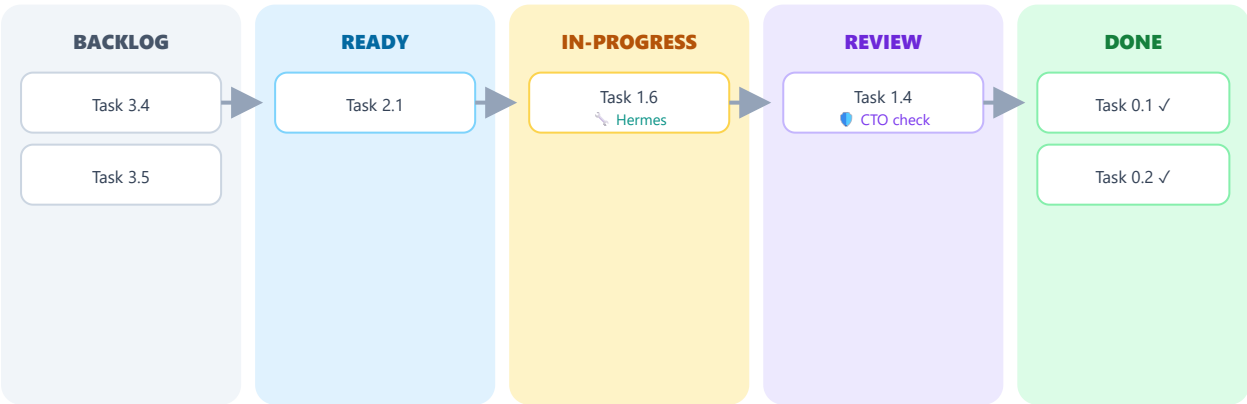
 Pull request: #8

Each colored pill is a **label**. Robots filter the board by these to find work meant for them.

What the four label groups mean

Label group	Question it answers	Possible values
state	What stage is this at?	backlog → ready → in-progress → review (also blocked , kickoff)
assigned:<model>	Which robot brain does it?	hermes-coder , claude-haiku , claude-sonnet , claude-opus
role:<role>	What kind of job is it?	coder , cto , architect
phase:<N>	Which chapter of the plan?	phase:0 ... phase:7

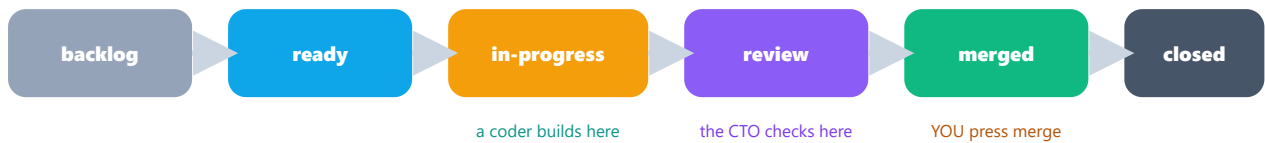
The board as a whole (a "kanban" board)



Tasks slide left → right as they progress. The Orchestrator is the one nudging them along.

6 · The life of a task

This is the heart of everything. Follow one task from "just an idea" to "done and approved."



1 Born

The Architect's `PLAN.md` is sliced into tasks. Each becomes a board note tagged `backlog` (waiting) or `ready` (good to go), plus its model + role + phase labels.

2 Unlocked

A task often waits on others ("Depends-on: #5"). The Orchestrator watches; when the things it needs are `closed`, it flips `backlog` → `ready`.

3 Handed out

The Orchestrator dispatches a `ready` task → `in-progress` and leaves a note: "dispatched to Hermes."

4 Built

The matching coder claims it (puts its name on it), makes a fresh branch, writes the files, and opens a **pull request** (a proposed change) that says "Closes #N." The task flips to `review`.

5 Checked

The Orchestrator opens a CTO review note. The CTO reads the change + the automatic tests (CI), then writes `VERDICT: APPROVED` or `CHANGES REQUIRED`.

6 Approved & merged

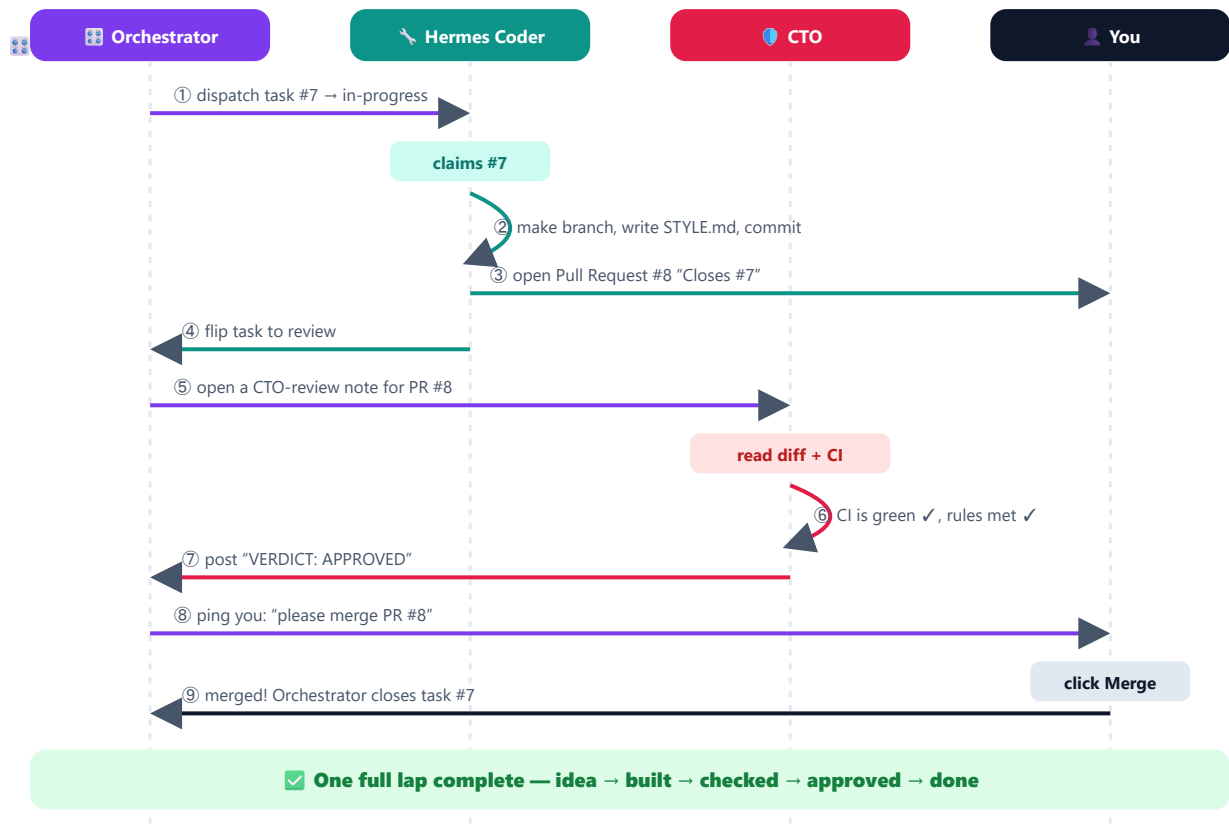
On `APPROVED`, the Orchestrator pings **you** to merge. You click merge; the Orchestrator closes the task. 🚩 If `CHANGES REQUIRED`, it bounces back to step 4.

WHAT "CI" MEANS

CI = a robot that automatically tries to **build the whole project** every time something changes. If the build breaks, CI turns **red** and the CTO will refuse to approve. If it builds fine, CI is **green**. Green CI is a hard requirement to merge.

7 · A real example, start to finish

Here's an actual task that ran: **"Write the book's style guide."** Watch it travel down the assembly line. The arrows show who does what, when.

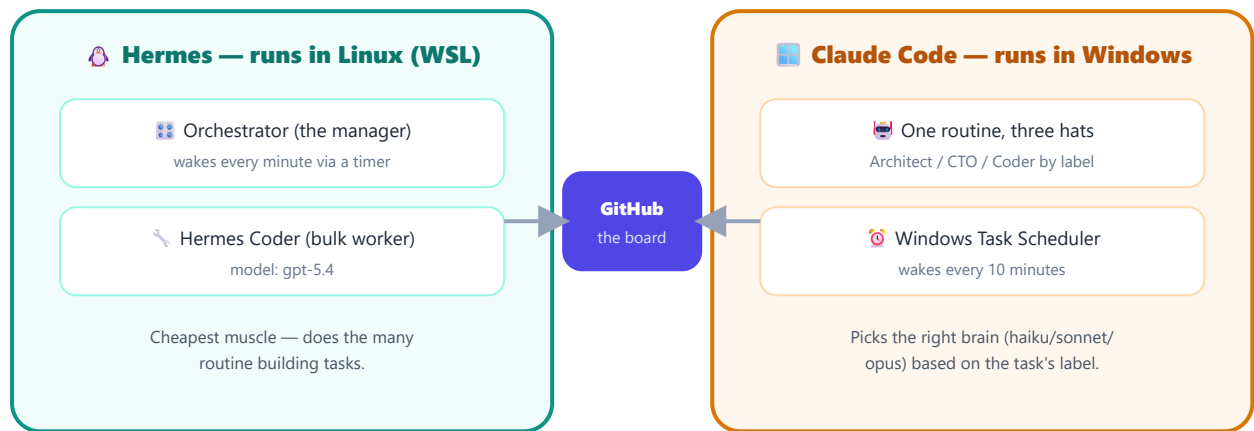


THIS REALLY HAPPENED

In the live system, task **#7** (the style guide) was built by the Hermes coder, opened as **Pull Request #8**, the CTO confirmed the build was green and approved it, and after a human merge the task closed automatically — exactly these nine steps.

8 · The two engines

The worker robots actually run on two different computers-within-your-computer. They never share memory — they only meet on the GitHub board.



Two separate worlds, one shared board. That's why everything important is written on GitHub — it's the only place they both can see.

WHY "STATELESS" MATTERS

Because each robot wakes up fresh with no memory, the board has to hold the truth. This is actually a superpower: if a robot crashes, the next one just reads the board and carries on. Nothing is lost.

9 · When things go wrong (the safety nets)

Real systems hit snags. Max Agency has automatic catches so a single hiccup doesn't freeze everything.



The build breaks (red CI)

A coder's change won't build. The CTO sees red CI and writes `CHANGES REQUIRED`. The task bounces back to `in-progress` and a coder tries again. Nothing broken ever reaches you.



A worker dies mid-job

If a robot grabs a task then crashes, the task could be stuck with its name on it forever. The Orchestrator notices "claimed, but no work after a while," **un-claims** it, and lets a fresh worker take over.



A genuine judgment call

If the CTO hits something only a human should decide, it writes `VERDICT: ESCALATE`. The Orchestrator forwards it to you (e.g. by Telegram) and pauses that thread.



Out of fuel (rate limits)

AI models have usage caps. If a robot hits its limit, its ticks fail quietly and simply retry later once the limit resets. The board waits patiently.

ANALOGY

Think of these like the safety features in a car: the seatbelt (human-only merge), the reversing camera (the CTO check), and the engine warning light (escalations). They don't drive the car for you — they stop small problems becoming crashes.

The three things only a human does

You, and only you...	Why it's reserved for a human
Approve the original plan	You're the one who knows what you actually want.
Merge finished work	The final, irreversible "yes this is good" needs a person.
Resolve escalations	These are the judgment calls the robots deliberately refuse to guess on.

10 · Glossary

Every bit of jargon in this guide, in plain words.

Word	Plain meaning
Repo (repository)	A project folder stored on GitHub, with full history of every change.
Issue	A note on the shared board describing one task. The robots' to-do items.
Label	A small colored tag on an issue. Says who/what/which-stage.
Branch	A private workspace copy where a coder makes changes without disturbing the main project.
Pull Request (PR)	A proposal: "here are my changes, please review and merge them in."
Merge	Accepting a pull request — folding the changes into the real project. Human-only.
CI (Continuous Integration)	A robot that auto-builds/tests the project on every change. Green = good, red = broken.
PLAN.md	The Architect's written blueprint: phases, tasks, and what depends on what.
Tick	One "wake up, look at the board, do one thing, go back to sleep" cycle of a robot.
Dispatch	The Orchestrator handing a ready task to a worker (moving it to in-progress).
Verdict	The CTO's decision on a piece of work: APPROVED, CHANGES REQUIRED, or ESCALATE.
Escalate	"A human needs to decide this." The robots stop and ask you.
Reclaim	Un-sticking a task whose worker died, so someone else can pick it up.
WSL	"Windows Subsystem for Linux" — a Linux computer running inside your Windows PC. Home of Hermes.
Hermes	The Linux-side engine running the Orchestrator and the bulk coder.
Claude Code	The Windows-side engine that can act as Architect, CTO, or the quality coder.
Stateless	Keeps no memory between ticks — reads everything fresh from the board each time.

11 · One-page cheat sheet



The five roles

- **Architect** — writes the plan
- **CTO** — reviews & approves
- **Orchestrator** — hands out work, chases progress
- **Hermes Coder** — cheap bulk builder (Linux)
- **Claude Coder** — careful builder (Windows)



The stages

- **backlog** → waiting on dependencies
- **ready** → good to start
- **in-progress** → a coder is on it
- **review** → CTO checking
- **merged** → **closed** → done ✓



Reading a label set

`in-progress` + `assigned:hermes-coder` +
`role:coder` + `phase:1`
= "Hermes should build this Phase-1 task; it's started."



Only you do these three

- Approve the plan
- Merge finished pull requests
- Answer escalations



The three places

- **Engine** — the robot toolkit (Max_Agency)
- **Copy** — Hermes's auto-mirror of it
- **Workshop** — your actual project repo



If it stalls

- Red CI → coder retries automatically
- Stuck task → Orchestrator reclaims it
- Out of fuel → waits for the limit to reset
- Big question → escalates to you

REMEMBER THIS ONE SENTENCE

You give a goal; a plan gets written and checked; a manager robot hands out tasks; worker robots build them; a reviewer robot checks each one; and you press the final button.

Everything else is just the plumbing that makes that loop run by itself, around the clock.